Image Processing Techniques - COMP...
PDF

hypothetically if i were to try to add bloom into my project to make it look like the lightsaber is on properly, with the white core and a coloured glow surrounding it, i do not have HDR setup at all and not much time, here is my main cpp file:

```
#include "scenebasic_uniform.h"

#include <cstdio>
#include <cstdlib>

#include <string>
using std::string;

#include <sstream>
#include <iostream>
using std::cerr;
using std::endl;

#include <GLFW/glfw3.h>
#include "helper/glutils.h"
#include "helper/texture.h"

using glm::vec3;
using glm::vec4;
using glm::mat3;
using glm::mat4;

SceneBasic_Uniform::SceneBasic_Uniform() : tPrev(0), angle(0.0f),
rotSpeed(glm::pi<float>() / 2.0f), plane(79.4f, 53.2f, 100, 100),
sky(100.0f) {
    C1Mesh = ObjMesh::load("media/C1-10P_obj.obj", true);
    //LightsaberMesh = ObjMesh::load("media/Lightsaber_01.obj", true);
    LightsaberMesh = ObjMesh::load("media/Lightsaber_03.obj", true);
    BladeMEsh = ObjMesh::load("media/cylinder.obj", true);
}

void SceneBasic_Uniform::initScene()
{
    compile();
    prog.use();
    glEnable(GL_DEPTH_TEST);

    model = mat4(1.0f);
    view = glm::lookAt(vec3(5.0f, 7.5f, 7.5f), vec3(0.0f, 0.75f, 0.0f),
vec3(0.0f, 1.0f, 0.0f));
    //view = glm::lookAt(camPos, camPos + camFront, camUp);

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        // Capture cursor
        glfwSetInputMode(win, GLFW_CURSOR, cursorCaptured ?
GLFW_CURSOR_DISABLED : GLFW_CURSOR_NORMAL);

        glfwGetCursorPos(win, &lastMouseX, &lastMouseY);
        firstMouse = true;
    }

    projection = mat4(1.0f);
    rotateModel = mat4(1.0f);
    rotateModel = glm::translate(rotateModel, vec3(4.0f, 0.26f, 5.0f));
    angle = 0.0f;


    //blinnPhongSpot stuff
    prog.setUniform("Spot.L", vec3(0.7f));
    prog.setUniform("Spot.La", vec3(0.5f));
    prog.setUniform("Spot.Exponent", 25.0f);
    prog.setUniform("Spot.Cutoff", glm::radians(20.0f));

    prog.setUniform("Light.L", vec3(0.5f));
    prog.setUniform("Light.La", vec3(0.25f));

    cubeTex =
Texture::loadCubeMap("media/texture/cube/space/space");
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);

    C1diffuseTexture =
Texture::loadTexture("media/texture/Chopper_BaseColor.png");
    C1normalMap =
Texture::loadTexture("media/texture/Chopper_Normal.png");
```

```
    //LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_01_lambert1_BaseColor
1.png");
    //LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_01_lambert1_Normal.pn
g");
    LSdiffuseTexture =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_BaseC
olor1.png");
    LSnormalMap =
Texture::loadTexture("media/texture/Lightsaber_03_exp_lambert1_Norm
al.png");

    BladeDiffuseTexture =
Texture::loadTexture("media/texture/white.jpg");
    WorkbenchDiffuseMap =
Texture::loadTexture("media/texture/workbench.png");

    // colours
    glm::vec3 WHITE(1.00f, 1.00f, 1.00f);
    glm::vec3 RED(1.00f, 0.00f, 0.00f);
    glm::vec3 GREEN(0.00f, 1.00f, 0.00f);
    glm::vec3 BLUE(0.00f, 0.45f, 1.00f);
    glm::vec3 YELLOW(1.00f, 1.00f, 0.00f);
    glm::vec3 ORANGE(1.00f, 0.50f, 0.00f);
    glm::vec3 MAGENTA(1.00f, 0.00f, 1.00f);

    bladeEmissive.use();
    bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 8.0f);
    bladeEmissive.setUniform("RimPower", 2.5f);
    bladeEmissive.setUniform("RimIntensity", 0.6f);


}

void SceneBasic_Uniform::compile()
{
    try {
        prog.compileShader("shader/basic_uniform.vert");
        prog.compileShader("shader/basic_uniform.frag");
        prog.link();
        prog.use();

        bladeEmissive.compileShader("shader/blade.vert");
        bladeEmissive.compileShader("shader/blade.frag");
        bladeEmissive.link();
        bladeEmissive.use();

        skyboxShader.compileShader("shader/skybox.vert");
        skyboxShader.compileShader("shader/skybox.frag");
        skyboxShader.link();
        skyboxShader.use();
    }
    catch (GLSLProgramException& e) {
        cerr << e.what() << endl;
        exit(EXIT_FAILURE);
    }
}

void SceneBasic_Uniform::update(float t)
{
    float deltaT = t - tPrev;
    if (tPrev == 0.0f)
        deltaT = 0.0f;
    tPrev = t;
    angle += 0.25f * deltaT;
    if (angle > glm::two_pi<float>())
        angle -= glm::two_pi<float>();

    GLFWwindow* win = glfwGetCurrentContext();
    if (win)
    {
        //blade toggle
        if (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS &&
!Q_Pressed) {
            bladeOn = !bladeOn;   // toggle once
        }
        Q_Pressed = (glfwGetKey(win, GLFW_KEY_Q) == GLFW_PRESS);
        // colour cycling
        if (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS && !E_Pressed)
        {
            bladeColourIndex = (bladeColourIndex + 1) %
(int)bladeColours.size();
```

```
            bladeEmissive.use();
            bladeEmissive.setUniform("BladeColour",
bladeColours[bladeColourIndex]);
        }
        E_Pressed = (glfwGetKey(win, GLFW_KEY_E) == GLFW_PRESS);
        //rotate hilt
        if (glfwGetKey(win, GLFW_KEY_W) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_S) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_UP) == GLFW_PRESS)
            hiltYawDeg -= hiltYawSpeed * deltaT;
        if (glfwGetKey(win, GLFW_KEY_DOWN) == GLFW_PRESS)
            hiltYawDeg += hiltYawSpeed * deltaT;

        if (hiltYawDeg > 360.0f) hiltYawDeg -= 360.0f;
        if (hiltYawDeg < 0.0f)   hiltYawDeg += 360.0f;
    }
    render();

}

void SceneBasic_Uniform::render()
{

    prog.use();

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);


    // BlinnPhong spotlight
    glm::vec4 lightPosWorld = glm::vec4(10.0f * cos(angle), 10.0f, 10.0f *
sin(angle), 1.0f);
    prog.setUniform("Spot.Position", glm::vec3(view * lightPosWorld));

    glm::vec3 targetWorld = glm::vec3(0.0f, 2.0f, 3.0f);
    glm::vec3 lightDirWorldSpot = glm::normalize(targetWorld -
glm::vec3(lightPosWorld));

    glm::vec3 lightDirView = glm::mat3(view) * lightDirWorldSpot;
    prog.setUniform("Spot.Direction", lightDirView);


    glm::vec3 lightDirWorld = glm::normalize(glm::vec3(-0.6f, -1.0f,
-0.4f));
    glm::vec3 lightDirViewSpace = glm::mat3(view) * -lightDirWorld;
    prog.setUniform("Light.Position", glm::vec4(lightDirViewSpace, 0.0f));


    // --- chopper to remove once happy with lighting
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, C1diffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, C1normalMap);

    prog.setUniform("Material.Ks", vec3(0.5f));
    prog.setUniform("Material.Ka", vec3(0.8f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, 2.0f, 5.0f));
    model = glm::rotate(model, glm::radians(45.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(2.0f));
    setMatrices();
    C1Mesh->render();


    // Lightsaber hilt
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, LSdiffuseTexture);

    glActiveTexture(GL_TEXTURE1);
    glBindTexture(GL_TEXTURE_2D, LSnormalMap);

    prog.setUniform("Material.Ks", vec3(0.25f));
    prog.setUniform("Material.Ka", vec3(0.15f));
    prog.setUniform("Material.Shininess", 80.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(-1.0f, 2.0f, 2.0f));
    model = glm::scale(model, vec3(1.5f));
    model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));
    model = glm::rotate(model, glm::radians(hiltYawDeg), vec3(0.0f, 1.0f,
0.0f));
```

```
    setMatrices();
    LightsaberMesh->render();


    // Blade
    if (bladeOn)
    {
        bladeEmissive.use();

        model = mat4(1.0f);
        model = glm::translate(model, vec3(10.0f, 2.1f, -4.0f));
        model = glm::scale(model, vec3(0.5f));
        model = glm::rotate(model, glm::radians(270.0f), vec3(1.0f, 0.0f,
0.0f));
        model = glm::rotate(model, glm::radians(300.0f), vec3(0.0f, 0.0f,
1.0f));

        setMatricesEmiss();
        BladeMEsh->render();
    }

    prog.use();
    // Workbench
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, WorkbenchDiffuseMap);

    prog.setUniform("Material.Ks", vec3(0.0f));
    prog.setUniform("Material.Ka", vec3(0.1f));
    prog.setUniform("Material.Shininess", 180.0f);

    model = mat4(1.0f);
    model = glm::translate(model, vec3(0.0f, -1.0f, 0.0f));
    model = glm::rotate(model, glm::radians(25.0f), vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, vec3(0.5f));
    setMatrices();
    plane.render();

    // Skybox stuff
    skyboxShader.use();
    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
    skyboxShader.setUniform("SkyboxTex", 0);

    glm::mat4 skyView = glm::mat4(glm::mat3(view));
    glm::mat4 skyMVP = projection * skyView;
    skyboxShader.setUniform("MVP", skyMVP);
    sky.render();
}

void SceneBasic_Uniform::setMatrices() {
    mat4 mv = view * model;
    prog.setUniform("ModelViewMatrix", mv);
    prog.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]), vec3(mv[1]),
vec3(mv[2])));
    prog.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::setMatricesEmiss() {
    mat4 mv = view * model;
    bladeEmissive.setUniform("ModelViewMatrix", mv);
    bladeEmissive.setUniform("NormalMatrix", glm::mat3(vec3(mv[0]),
vec3(mv[1]), vec3(mv[2])));
    bladeEmissive.setUniform("MVP", projection * mv);
}

void SceneBasic_Uniform::resize(int w, int h) {
    glViewport(0, 0, w, h);
    width = w;
    height = h;
    projection = glm::perspective(glm::radians(70.0f), (float)w / h, 0.3f,
100.0f);
}
```

ive also attached the lab sheet detailing how to add hdr, i would go through it properly if i had time but i dont have the time to go through it fully and understand it
do you need to see any of my other files (like my vert or frag files etc) before you help me decide if its worth attempting now, or if i should stick to what i have and do tweaks to improve instead of adding a new type of rendering

Thought for 13s ›

You don't *need* to show me your other files just to decide whether it's worth attempting right now — your **C++ render flow** already tells me the main thing: you're doing **single-pass forward rendering straight to**

**the default framebuffer**, and bloom (the "proper" kind) means **introducing an off-screen render target + fullscreen composite** (at minimum). That's a structural change, not a tweak.

From your lab sheet, the bloom workflow they teach is explicitly **multi-pass and HDR-based** (render scene to an HDR texture, bright-pass, blur, then composite with tone mapping) Image Processing Techniques - C... . With "not much time", that's the bit that makes bloom risky.

## Is it worth attempting bloom now?

### If you try to follow the lab bloom exactly: high risk

Reasons (based on the lab's approach):

- You must add **FBO(s), float textures (RGB32F), and a fullscreen quad** plumbing first Image Processing Techniques - C... .
- Bloom there is **5 passes** (HDR render → bright-pass → blur H → blur V → composite + tone map) Image Processing Techniques - C... .
- You'll also be integrating a new shader path that uses a `Pass` uniform and swaps behaviour per-pass (again: more plumbing, more "where did my scene go?" debugging time) Image Processing Techniques - C... .

If your deadline is tight, this is the kind of change that can eat hours just on "black screen / wrong attachment / wrong viewport / depth test / sampler state".

### A "cheap bloom" variant: maybe, if you keep it tiny

If you *really* want glow but can't afford HDR + tone mapping:

**Option A (fastest, no post-process): "mesh glow shell"**

- Render the blade normally (your current emissive shader).
- Render it a **second time**, slightly **scaled up** (e.g. 1.03–1.10), with:
  - **additive blending** (`glEnable(GL_BLEND); glBlendFunc(GL_ONE, GL_ONE);`)
  - **depth write off** (`glDepthMask(GL_FALSE);`) but keep depth test on
  - fragment shader outputs a **soft rim** (you already have rim controls) with **lower alpha** and maybe a wider rim.
    This gives you a coloured halo around the white core without any framebuffer work. It won't bloom onto the environment, but it reads like a lightsaber in motion and is very controllable.

**Option B (still simpler than HDR bloom): LDR bright mask → blur → add**

- Add **one small FBO** (like width/8, height/8) that you render **only the blade glow** into (black background).
- Blur it (two-pass Gaussian, like the lab's blur section) then **add** it onto the final scene.
  This gets you an actual screen-space glow without full HDR/tone mapping. Still some plumbing, but far less than full lab bloom.

If you want "white core + coloured glow", Option A often sells it surprisingly well, and it's the least likely to explode your project late on.

## Do I need your shaders to help you pick a direction?

Not to decide. But **yes** if you want me to give you exact code changes.

If you go:

- **Option A (glow shell):** I'd want to see **blade.frag** (and blade.vert if it does anything unusual), because the whole trick is controlling rim falloff / alpha cleanly.
- **Option B or full bloom:** I'd need:
  - whatever shader you'll use for the fullscreen quad (you can base it on the lab's post-process approach),
  - and a quick look at how your framework handles VAOs/programs (but your file already hints you're using the lab-style `GLSLProgram` wrapper).

## My recommendation given your constraints

- If this is close to submission: **don't attempt full HDR bloom right now**. It's too many moving parts for the time you've got. The lab bloom section is a full pipeline exercise, not a quick bolt-on Image Processing Techniques - C... .
- Do **Option A (glow shell)** and spend your time making the blade read well:
  - core = near-white, clamp to 1.0
  - glow pass = blade colour, additive, rim-heavy, slightly larger scale
  - tweak your `Intensity`, `RimPower`, `RimIntensity` to avoid it looking like a plastic tube.

If you paste your current `blade.frag` (and tell me whether you're happy to render the blade twice), I can show you exactly what to change and what OpenGL state to toggle for the glow pass.

Sources

lets go for the option A of the glow shell as it sounds like its going to be the simplest way to get the lightsaber vibe.

heres the blade.frag:
#version 460

```
in vec3 vNormalVS;
in vec3 vPosVS;

layout (location = 0) out vec4 FragColor;

// Set from C++
uniform vec3 BladeColour;     // (1,1,1) for white
uniform float Intensity;      // brightness, e.g. 5..20
uniform float RimPower;       // e.g. 2.0..4.0
uniform float RimIntensity;   // e.g. 0.0..1.0 (extra edge pop)

void main()
{
    vec3 N = normalize(vNormalVS);
    vec3 V = normalize(-vPosVS);              // camera is origin in view-
space

    float rim = pow(1.0 - max(dot(N, V), 0.0), RimPower);

    // Pure emissive core + optional rim boost
    vec3 col = BladeColour * Intensity;
    col += BladeColour * (rim * RimIntensity * Intensity);

    FragColor = vec4(col, 1.0);
}

and heres the blade.vert:
#version 460

layout (location = 0) in vec3 VertexPosition;
layout (location = 1) in vec3 VertexNormal;

out vec3 vNormalVS;
out vec3 vPosVS;

uniform mat4 ModelViewMatrix;
uniform mat3 NormalMatrix;
uniform mat4 MVP;

void main()
{
    vNormalVS = normalize(NormalMatrix * VertexNormal);
    vPosVS = (ModelViewMatrix * vec4(VertexPosition, 1.0)).xyz;

    gl_Position = MVP * vec4(VertexPosition, 1.0);
}
```

Nice — your blade shaders are already basically perfect for a glow-shell approach. We just need two things:

1. **Render the blade twice** (core pass, then glow pass) with different uniforms + OpenGL state.
2. Make the **glow pass fade out** (alpha) so additive blending doesn't just nuke the screen.

Below is a minimal, low-risk way to do it.

---

### 1) Update `blade.frag` to support a controllable alpha for the glow pass

Right now you always output `alpha = 1.0`. For the glow shell we want alpha based on rim (edge = stronger, centre = weaker).

Replace your `blade.frag` with:

```
#version 460

in vec3 vNormalVS;
in vec3 vPosVS;

layout (location = 0) out vec4 FragColor;

// Set from C++
uniform vec3 BladeColour;
uniform float Intensity;
uniform float RimPower;
uniform float RimIntensity;

// New (for glow shell control)
uniform float Alpha;          // overall alpha scale, e.g. 0.15..0.6
uniform float CoreAlpha;      // base alpha even at centre, e.g. 0.0..0.2

void main()
{
    vec3 N = normalize(vNormalVS);
    vec3 V = normalize(-vPosVS); // camera at origin in view-space

    float ndv = max(dot(N, V), 0.0);
    float rim = pow(1.0 - ndv, RimPower);

    // Emissive colour (core + extra rim pop)
```

```
    vec3 col = BladeColour * Intensity;
    col += BladeColour * (rim * RimIntensity * Intensity);
```

glsl

```
    FragColor = vec4(col, a);
}
```

No change needed in `blade.vert`.

---

## 2) Render the blade twice in C++

### A) Core pass (white core)

- No blending
- Normal depth test + depth write
- Intensity high, rim subtle
- Alpha irrelevant (but we still set it to 1)

### B) Glow shell pass (coloured halo)

- **Additive blending**
- **Depth test ON**, but **depth write OFF** (so it doesn't mess the rest)
- Slightly bigger scale (like `1.06f`)
- Lower intensity than core, rim stronger
- Alpha something like 0.25–0.5, CoreAlpha 0

Here's a drop-in edit for your `render()` blade section.

Replace your current blade block with this:

```
// Blade
if (bladeOn)
{
    // Shared transform basis for both passes
    glm::mat4 bladeBase = mat4(1.0f);
    bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.0f));
    bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));

    // ---------- Pass 1: White core ----------
    bladeEmissive.use();

    glDisable(GL_BLEND);
    glEnable(GL_DEPTH_TEST);
    glDepthMask(GL_TRUE);

    model = bladeBase;
    model = glm::scale(model, vec3(0.50f)); // your original scale

    setMatricesEmiss();

    bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f)); // white core
    bladeEmissive.setUniform("Intensity", 10.0f);
    bladeEmissive.setUniform("RimPower", 3.0f);
    bladeEmissive.setUniform("RimIntensity", 0.25f);

    // new uniforms (safe even if you forget to use them in shader later)
    bladeEmissive.setUniform("Alpha", 1.0f);
    bladeEmissive.setUniform("CoreAlpha", 1.0f);

    BladeMEsh->render();

    // ---------- Pass 2: Glow shell (coloured halo) ----------
    bladeEmissive.use();

    glEnable(GL_BLEND);
    glBlendFunc(GL_ONE, GL_ONE);   // additive glow
    glDepthMask(GL_FALSE);         // don't write depth (but depth test still blocks behind-hilt

    model = bladeBase;
    model = glm::scale(model, vec3(0.50f * 1.08f)); // slightly larger shell

    setMatricesEmiss();

    bladeEmissive.setUniform("BladeColour", bladeColours[bladeColourIndex]); // coloured glow
    bladeEmissive.setUniform("Intensity", 2.5f);
    bladeEmissive.setUniform("RimPower", 2.0f);
    bladeEmissive.setUniform("RimIntensity", 1.0f);

    bladeEmissive.setUniform("Alpha", 0.45f);
    bladeEmissive.setUniform("CoreAlpha", 0.0f);
```

C++

```
    // restore state for rest of scene
    glDepthMask(GL_TRUE);
    glDisable(GL_BLEND);
}
```

That's it structurally.

---

### 3) Quick tuning suggestions (so it doesn't look rubbish)

Start with these values (they're intentionally conservative):

**Core pass**

- `Intensity = 8..14`
- `RimPower = 3..5` (higher = tighter edge)
- `RimIntensity = 0.1..0.35`

**Glow pass**

- shell scale = `1.05..1.12`
- `Intensity = 1.5..4.0`
- `RimPower = 1.6..2.6`
- `Alpha = 0.25..0.65`
- `CoreAlpha = 0.0` (usually)

If the glow looks like a fat tube: increase `RimPower` a bit and/or reduce the shell scale.

If it's barely visible: raise `Intensity` first, then `Alpha`.

---

### 4) One important gotcha in your current code

Your blade is currently not attached to the hilt position/rotation (hilt is at `(-1,2,2)` but blade is at `(10,2.1,-4)`).
Glow will still work, but visually it'll feel "wrong" unless the blade sits in the emitter. If you want, paste the exact transform you *want* for the blade (or we can reuse the hilt matrix and add an offset) and I'll show a clean way to parent it.

---

If you paste your current `SceneBasic_Uniform` header (just the member vars for blade/hilt transforms + your `bladeColours` vector setup), I can also show a tidy way to keep core/glow parameters together without making your render() messy.

okay so it kinda works, but the blade colours only show where the blade overlaps with the hilt, i want the colour to be visible around the blade fully, this doesnt work how i was hoping

Thought for 35s ›

Yep — what you're seeing is **depth test doing its job a bit too well**.

Right now you render:

1. **Core pass** → writes depth
2. **Glow shell pass** → is *behind* the core (expanded mesh = often further away), so **fails the depth test almost everywhere**.

That's why you only see glow in weird spots (like where it intersects/overlaps the hilt).

### Fix option (fast + works): draw glow without depth testing

This guarantees the halo shows around the blade silhouette.

In your **glow pass**, change the state to:

```cpp
// ---------- Pass 2: Glow shell (coloured halo) ----------
bladeEmissive.use();

glEnable(GL_BLEND);
glBlendFunc(GL_ONE, GL_ONE);
```

C++

...and after the glow pass, restore:

```cpp
glEnable(GL_DEPTH_TEST);
```

C++

That should immediately make the glow visible around the whole blade.

### Downside

The glow can appear "on top" of the hilt/other objects (because depth is off). If you can live with that for a tight deadline, it's the simplest win.

---

### Fix option (still simple, looks better): use front-face culling + depth test ON

This is the classic "outline shell" trick. It keeps depth test, but only draws the **backfaces** of the expanded shell, which usually gives a clean halo around the silhouette.

For glow pass:

```cpp
bladeEmissive.use();

glEnable(GL_BLEND);
glBlendFunc(GL_ONE, GL_ONE);

glEnable(GL_DEPTH_TEST);
glDepthMask(GL_FALSE);

// Only draw backfaces of the expanded mesh
glEnable(GL_CULL_FACE);
```

C++

```cpp
// (keep your scaled-up model here)
```

Then restore:

```cpp
glCullFace(GL_BACK);
glDisable(GL_CULL_FACE);
```

C++

If this works in your camera angle, it'll look *much* more "wrapped" around the blade without bleeding through everything.

---

### If you want the cleanest result (still not too bad): stencil mask (optional)

If the "depth off" version looks wrong and the culling version still doesn't show enough halo, stencil is the proper middle ground:

- render the **core** and write stencil = 1
- render the **glow** with stencil test "NOT EQUAL 1" and depth off

But I won't dump that on you unless you want it — it's a bit more setup-y.

---

### Quick check

Your blade is a **cylinder**. If your cylinder has **no caps** or weird normals, the rim term might behave oddly, but since you're getting glow *only sometimes*, this really screams **depth rejection**, not shader maths.

---

Tell me which of these you want:

- **A)** depth off (fastest)
- **B)** front-face culling (usually best-looking with minimal effort)

...and I'll give you the exact edited blade block in your `render()` with the right state ordering (so it doesn't break your skybox/workbench afterwards).

---

i think you misunderstood.
what you gave me just now made it so the blade was visible through the hilt. this was NOT what i asked for or wanted whatsoever.
the issue i told you about was that the colour is supposed to wrap around the blade only, so that it looks as if the blade has the coloured glow regardless of the angle youu view it at, this must not bleed through the hilt, its supposed to look as if the blade is being emitted from the hilt, not through it.

Thought for 14s ›

Got you — you want the glow to **hug the blade silhouette**, but **never appear through/over the hilt**, and also **not wash over the white core**.

The clean way to do that (without HDR/bloom) is:

- **Depth test stays ON** (so the hilt blocks the blade/glow properly)
- Use the **stencil buffer** so the glow only draws **outside** the blade core silhouette

This is the standard "outline/glow" trick for exactly your requirement.

---

### 0) One prerequisite: you must actually have a stencil buffer

If your GLFW window wasn't created with stencil bits, stencil testing will do nothing.

Where you create the window (usually in `main` before `glfwCreateWindow`), add:

```
C++
```

You can also sanity check once at runtime:

```
int s = 0;
C++
```

If it prints `0`, stencil won't work.

---

### 1) Enable stencil testing (once)

In `initScene()` after you enable depth:

```
glEnable(GL_DEPTH_TEST);

C++
```

---

### 2) Clear stencil every frame

In `render()` change your clear call to include stencil:

```
C++
```

---

### 3) Blade drawing: core writes stencil = 1, glow draws where stencil != 1

Replace your blade block with this version (core first, then glow):

```
// Blade
if (bladeOn)
{
    glm::mat4 bladeBase = mat4(1.0f);
    bladeBase = glm::translate(bladeBase, vec3(10.0f, 2.1f, -4.0f));
    bladeBase = glm::rotate(bladeBase, glm::radians(270.0f), vec3(1.0f, 0.0f, 0.0f));
    bladeBase = glm::rotate(bladeBase, glm::radians(300.0f), vec3(0.0f, 0.0f, 1.0f));

    // -------- Pass 1: White core (writes stencil = 1 where the blade is) --------
    bladeEmissive.use();

    glDisable(GL_BLEND);
    glEnable(GL_DEPTH_TEST);
    glDepthMask(GL_TRUE);

    // Stencil: always write 1 where fragments pass depth test
    glStencilMask(0xFF);
    glStencilFunc(GL_ALWAYS, 1, 0xFF);
    glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);

    model = bladeBase;
    model = glm::scale(model, vec3(0.50f));
    setMatricesEmiss();

    bladeEmissive.setUniform("BladeColour", glm::vec3(1.0f));
    bladeEmissive.setUniform("Intensity", 10.0f);
    bladeEmissive.setUniform("RimPower", 3.0f);
    bladeEmissive.setUniform("RimIntensity", 0.25f);

    bladeEmissive.setUniform("Alpha", 1.0f);
    bladeEmissive.setUniform("CoreAlpha", 1.0f);

    BladeMEsh->render();

    // -------- Pass 2: Coloured glow shell (only OUTSIDE the core silhouette) --------
    glEnable(GL_BLEND);
    glBlendFunc(GL_ONE, GL_ONE);

    // Don't write depth for glow, but DO depth test so hilt blocks it
    glDepthMask(GL_FALSE);

    // Stencil: only draw where stencil != 1 (i.e., outside the core)
    glStencilMask(0x00);
    glStencilFunc(GL_NOTEQUAL, 1, 0xFF);
    glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);

    // Optional but usually improves the "wrap" look:
    // render only backfaces of the enlarged shell so it appears as an outline
    glEnable(GL_CULL_FACE);
    glCullFace(GL_FRONT);
```

```
    model = bladeBase;
    model = glm::scale(model, vec3(0.50f * 1.08f));
    setMatricesEmiss();

    bladeEmissive.setUniform("BladeColour", bladeColours[bladeColourIndex]);
    bladeEmissive.setUniform("Intensity", 2.5f);
    bladeEmissive.setUniform("RimPower", 2.0f);
    bladeEmissive.setUniform("RimIntensity", 1.0f);

    bladeEmissive.setUniform("Alpha", 0.55f);
    bladeEmissive.setUniform("CoreAlpha", 0.0f);

    BladeMEsh->render();

    // -------- Restore state --------
```

C++

```
    glDepthMask(GL_TRUE);
    glDisable(GL_BLEND);

    glStencilMask(0xFF);
    glStencilFunc(GL_ALWAYS, 0, 0xFF);
    glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
}
```

### Why this fixes your exact complaint

- The glow **cannot draw on top of the blade core**, because stencil blocks it where the core was drawn.
- The glow **still respects depth**, so it won't "bleed through" the hilt (or anything else).
- The glow shows around the blade silhouette from any angle (that's the whole point of the enlarged shell + stencil).

---

### If you still only see glow in odd patches

That usually means one of these:

1. **No stencil buffer** (check stencil bits).
2. Your cylinder mesh has odd topology / normals; try disabling cull face for the glow pass (leave stencil logic as-is).
3. Your blade transform doesn't actually line up with the hilt (so you think it's "through the hilt" but it's really just placed inside it).

If you want, paste the bit where you create the GLFW window (just that function), and I'll point out exactly where to slot the stencil hint so you don't have to hunt for it.

so the issue was that this method needs a model rendered behind it visually before the blade itself. this was a detail you didnt mention to me causing this confusion.
is it possible to have it so that i dont have to have something behind it rendered first? for example behind it would be the skybox

Thought for 20s

You *don't* need anything "behind it" for the stencil glow to work — the glow pass adds colour to **whatever is already in the colour buffer**, and the colour buffer always has *something* in it after `glClear(...)` (even if it's just the clear colour).

What *can* make it **look** like it "needs something behind it" is **render order / skybox overwriting**, or **drawing the skybox last in a way that ignores depth**.

### The usual culprit in your current render order

You draw the skybox **at the very end**:

```
// ... draw everything ...
// Skybox stuff
skyboxShader.use();
```

C++

If your skybox draw is not correctly depth-tested (or it writes depth / uses a depth func that passes everywhere), it can simply paint over your blade/glow. That makes it feel like "the glow only works when other geometry is behind it", because the hilt/other meshes might still win due to depth, but the glow (blended + depth-mask off) gets visually wiped out.

### Fix: draw the skybox first (fastest, robust)

Do this near the top of `render()`, right after the clear, before any other geometry:

```
// Clear
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT | GL_STENCIL_BUFFER_BIT);
```

```cpp
// Draw skybox FIRST
skyboxShader.use();
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_CUBE_MAP, cubeTex);
skyboxShader.setUniform("SkyboxTex", 0);

glm::mat4 skyView = glm::mat4(glm::mat3(view));
glm::mat4 skyMVP = projection * skyView;
skyboxShader.setUniform("MVP", skyMVP);

// Important skybox state
glDepthMask(GL_FALSE);          // don't write depth
glDepthFunc(GL_LEQUAL);         // pass when depth == 1.0 too (common for skybox)
sky.render();
```

C++

```cpp
glDepthFunc(GL_LESS);
glDepthMask(GL_TRUE);
```

Then render your opaque stuff (hilt, plane, chopper), then your blade core + stencil glow.

That guarantees the skybox is "behind everything", and nothing gets overwritten later.

### If you really want to keep skybox last

You *can*, but then you must make sure it can't overwrite existing pixels:

- depth test must be ON
- depth func should be `GL_LEQUAL`
- depth writes OFF
- and you must not disable depth testing for the skybox draw

Same state block as above, just keep it at the end. If you had depth disabled for skybox, it will paint over everything.

---

### One more thing to sanity-check

Make sure you're clearing stencil every frame:

ChatGPT 5.2 Thinking

Share

If you don't clear stencil, you can get "random" areas where the glow shows/doesn't.

---

If you paste your `skybox.vert/frag` (or just tell me if you change depth state anywhere for the skybox), I can tell you exactly why it's overwriting and the minimal fix.

okay thank you, now could you help me with a quick overview for how i should edit things so i can tweak them to look better, falloff, colour size etc

Yep — with the **stencil + shell glow** setup, the look is basically controlled by 4 knobs:

1. **Shell scale** (how "thick" the glow is)
2. **Rim shaping** (how the glow concentrates towards the edge)
3. **Intensity/Alpha** (how bright it is, and how quickly it falls off)
4. **Colour choice** (your blade colour itself)

Here's a practical checklist you can tweak quickly.

---

## Core vs Glow: what each pass should do

### White core pass (blade itself)

Goal: solid white tube with only a *little* edge shaping.

- `BladeColour = vec3(1)`
- `Intensity` = **8–14**
- `RimPower` = **3–6** (higher = tighter rim)
- `RimIntensity` = **0.1–0.35**

If it looks "flat/plastic": increase `RimIntensity` a bit.
If it looks "outlined": increase `RimPower` or lower `RimIntensity`.

---

### Glow pass (shell)

Goal: colour visible around silhouette, not filling the centre.

- **Shell scale** = **1.04–1.12** (start ~1.07)
- `BladeColour = bladeColours[index]`
- `Intensity` = **1.5–4.0**

- `RimPower` = **1.4–2.6**
- `RimIntensity` = **0.8–1.5**
- `Alpha` = **0.2–0.7**
- `CoreAlpha` = **0.0–0.08** (usually 0)

---

### What each knob *actually* changes (so you know what to touch)

#### 1) Glow thickness: shell scale

In C++ (glow pass):

```
C++
```

- Bigger `glowScale` → thicker halo
- Too big → starts looking like a second blade

**Start points**

- Thin Jedi-ish: `1.05–1.08`
- Thicker/glossier: `1.09–1.12`

---

#### 2) Falloff / edge focus: RimPower

In your shader, rim is:
`rim = pow(1 - dot(N,V), RimPower)`

- Higher `RimPower` → glow concentrates into a thin edge band
- Lower `RimPower` → glow spreads more across the surface

**Useful ranges**

- Core pass: `3–6`
- Glow pass: `1.4–2.6`

---

#### 3) Brightness: Intensity vs Alpha

You're using additive blending for glow, so:

- `Intensity` affects the *colour value* added
- `Alpha` affects how much of that you add *based on rim*

If your glow is nuking the scene:

- lower `Intensity` first, then lower `Alpha`

If your glow is barely visible:

- raise `Intensity` first (to ~3–4), then `Alpha` (to ~0.6)

---

#### 4) "Size" of coloured region around the core: Alpha + RimPower

A really good combo trick:

- Want **bigger coloured aura**?
  ↓ `RimPower` a bit AND ↑ `Alpha` a bit
- Want **tight aura**?
  ↑ `RimPower` AND ↓ `Alpha`

---

### Quick recipes (copy these values)

#### "Classic lightsaber" (tight glow)

**Core**

- `Intensity 12`
- `RimPower 5`
- `RimIntensity 0.2`

**Glow**

- `Scale 1.06`
- `Intensity 2.5`
- `RimPower 2.4`
- `RimIntensity 1.0`
- `Alpha 0.45`
- `CoreAlpha 0.0`

---

#### "Stronger cinematic glow" (bigger aura)

**Glow**

- `Scale 1.10`
- `Intensity 3.5`
- `RimPower 1.7`

- `RimIntensity 1.2`
- `Alpha 0.65`

---

### “Subtle” (barely there)

**Glow**

- `Scale 1.05`
- `Intensity 1.8`
- `RimPower 2.6`
- `Alpha 0.25`

---

### Tiny improvement that makes it look less “gamey”

Right now glow is purely rim-based. If you want it to feel softer, add a *secondary* falloff term, like:

- keep rim for silhouette
- but multiply the glow by something like `(0.5 + 0.5 * rim)` or clamp it

If you want, I can give you a 2-line tweak to your `a` calculation to make the fade smoother (without changing your pipeline).

---

### Practical workflow (fast)

1. Lock the core so it looks like a solid white tube.
2. Adjust only **glow scale** until thickness feels right.
3. Adjust **RimPower** until it wraps nicely at different angles.
4. Adjust **Intensity/Alpha** last to avoid chasing your tail.

---

If you paste your current glow-pass numbers (scale/intensity/rimpower/alpha), I’ll suggest a tighter set tailored to what you’re seeing (too thick, too bright, too faint, etc.).